

LINQ ToLookup Method in C# with Examples

In this article, I will discuss the **LINQ ToLookup Method in C#** with Examples. Please read our previous article, discussing the [LINQ GroupBy Method with Multiple Keys in C#](#) with Examples. As part of this article, we will discuss the following pointers.

1. [What is the LINQ ToLookup Method in C#?](#)
2. [Examples of using the ToLookup Method in C# using both Method and Query Syntax.](#)
3. [How Do We Use the ToLookup method with the OrderBy method?](#)
4. [How Do We Use the ToLookup method in C# with Multiple Keys?](#)
5. [What is the difference between ToLookup and GroupBy Methods in Linq?](#)
6. [When Should We Use the LINQ ToLookup Method in C#?](#)
7. [Multiple Real-Time Examples of LINQ ToLookup Method.](#)

What is the LINQ ToLookup Method in C#?

The ToLookup method in LINQ (Language Integrated Query) is a method used to create a Lookup<TKey, TElement> from an IEnumerable<T>. A Lookup<TKey, TElement> is similar to a Dictionary<TKey, TValue>, but whereas a dictionary maps a key to a single value, a lookup maps a key to a collection of values. This makes Lookup useful in scenarios where you want to group elements of a sequence by a certain key and have quick access to these groups later on. The basic syntax for ToLookup looks something like this:

```
var lookup = source.ToLookup(keySelector, elementSelector);
```

Here,

- **source:** The data source to create the lookup from. It's an IEnumerable<T>.
- **keySelector:** A function to extract a key from each element.
- **elementSelector:** A function to map each source element to an element in the Lookup. This parameter is optional; if not specified, the element itself is used.

Note: The LINQ **ToLookup** Method exactly does the same thing as the LINQ **GroupBy** Method. The only difference between these two methods is that the GroupBy method uses Deferred Execution, whereas the ToLookup method uses Immediate Execution. Before proceeding further, let us first understand **what Deferred Execution and Immediate Execution are in LINQ**.

What is LINQ Deferred Execution?

In the case of Deferred Execution, the LINQ Query is not executed at the point of its declaration. That means it doesn't execute by itself when we write a LINQ query. It executes only when we access the query results. So, here, the execution of the query is deferred until the query variable is iterated over using a for-each loop. For example, Select, SelectMany, Where, Take, Skip, etc., belong to the Deferred or Lazy Operators Category.

What is LINQ Immediate Execution?

In the case of Immediate Execution, the LINQ query is executed at the point of its declaration. So, it forces the query to execute and gets the result immediately, for example. Count, Average, Min, Max, First, Last, ToArray, ToList, etc., belongs to the Immediate or Greedy Operators category. Please read the following article to understand what Deferred and Immediate Execution are in Linq Queries in detail with examples.

LINQ ToLookup Method in C# with Examples

Let us understand how to use the LINQ ToLookup Method in C# using both Method and Query Syntax with Examples. We will use the following Student class to understand the ToLookup Method. So, create a class file named **Student.cs** and copy and paste the following code. The following Student class contains five properties: ID, Name, Gender, Branch, and Age. This class also has one method called GetStudents(), which returns a list of all students and will be our data source.

```
using System.Collections.Generic;
namespace ToLookupDemo
{
    public class Student
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public string Gender { get; set; }
        public string Branch { get; set; }
        public int Age { get; set; }
        public static List<Student> GetStudents()
        {
            return new List<Student>()
            {
                new Student { ID = 1001, Name = "Preety", Gender = "Female", Branch = "CSE", Age = 20 },
                new Student { ID = 1002, Name = "Snurag", Gender = "Male", Branch = "ETC", Age = 21 },
                new Student { ID = 1003, Name = "Pranaya", Gender = "Male", Branch = "CSE", Age = 21 },
                new Student { ID = 1004, Name = "Anurag", Gender = "Male", Branch = "CSE", Age = 20 },
                new Student { ID = 1005, Name = "Hina", Gender = "Female", Branch = "ETC", Age = 20 },
                new Student { ID = 1006, Name = "Priyanka", Gender = "Female", Branch = "CSE", Age = 21 },
                new Student { ID = 1007, Name = "santosh", Gender = "Male", Branch = "CSE", Age = 22 },
                new Student { ID = 1008, Name = "Tina", Gender = "Female", Branch = "CSE", Age = 20 },
                new Student { ID = 1009, Name = "Celina", Gender = "Female", Branch = "ETC", Age = 22 },
                new Student { ID = 1010, Name = "Sambit", Gender = "Male", Branch = "CSE", Age = 21 }
            };
        }
    }
}
```

Grouping the Students Based on Branch using LINQ ToLookup Method in C#

Now, we want to group the students based on their Branch. For a better understanding, please look at the following example, which exactly does the same using the ToLookup Method with both Method and Query Syntax. The following example organizes the students into groups based on their branch (i.e., the branch will act as the key). It also means that students with the same branch will be stored in the same group, where each group has a key and the corresponding student collection. Here, the key will be the Branch, and the collection will be the students belonging to that particular branch.

```
using System;
using System.Linq;
namespace ToLookupDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Grouping the Students Based on Branch using ToLookup Method
            //Using Method Syntax
            var GroupByMS = Student.GetStudents().ToLookup(s => s.Branch);
            //Using Query Syntax
            var GroupByQS = (from std in Student.GetStudents()
                             select std).ToLookup(x => x.Branch);
            //It will iterate through each group
            foreach (var group in GroupByMS)
            {
                Console.WriteLine(group.Key + " : " + group.Count());
                //Iterate through each student of a group
                foreach (var student in group)
                {
                    Console.WriteLine("    Name :" + student.Name + ", Age: " +
+ ", Gender :" + student.Gender);
                }
            }
            Console.Read();
        }
    }
}
```

Output:

```

CSE : 6
Name :Preety, Age: 20, Gender :Female
Name :Pranaya, Age: 21, Gender :Male
Name :Anurag, Age: 20, Gender :Male
Name :Priyanka, Age: 21, Gender :Female
Name :santosh, Age: 22, Gender :Male
Name :Tina, Age: 20, Gender :Female
ETC : 4
Name :Snurag, Age: 21, Gender :Male
Name :Hina, Age: 20, Gender :Female
Name :Celina, Age: 22, Gender :Female
Name :Sambit, Age: 21, Gender :Male

```

Note: Each group has a key, and you can access the value of the key by using the key property. Along the same line, you can also use the count property to check how many elements are in that group. Again, you can access all the group elements shown in the above example using a for each loop.

Key Features of LINQ ToLookup Method in C#:

- **Multiple Values Per Key:** ToLookup allows multiple values for each key, which is the primary difference from using a Dictionary<TKey, TValue>.
- **Immediate Execution:** Unlike many other LINQ methods that use deferred execution, ToLookup immediately executes and consumes the source sequence to build the Lookup.
- **Lookup vs. GroupBy:** While ToLookup and GroupBy both group elements by key, ToLookup immediately executes and returns a Lookup<TKey, TElement>, whereas GroupBy uses deferred execution and returns an IEnumerable<IGrouping<TKey, TElement>>.

Grouping Students by Gender in Descending Order, Names in Ascending Order in Each Group

Let us see an example of Understanding the LINQ ToLookup Method with the following requirements.

1. First, Grouping the Students by Gender.
2. Then, sort the Groups in Descending Order, i.e., sorting the Gender in Descending Order.
3. Finally, Sort the Student Names in each group in Ascending Order.

In the following example, we group the students by Gender using the LINQ ToLookup Method. But here, we first sort the data by Gender in Descending Order and then sort the student in each group by their name in Ascending Order.

```

using System;
using System.Linq;
namespace ToLookupDemo
{
    class Program
    {
        static void Main(string[] args)

```

```

{
    //Using Method Syntax
    //First Group the Data by Gender using ToLookup Method
    var GroupByMS = Student.GetStudents().ToLookup(s => s.Gender)
    //Then Sorting the data based on key in Descending
    .OrderByDescending(c => c.Key)
    .Select(std => new
    {
        Key = std.Key,
        //Sorting the Students in Each Group based on
Ascending order
        Students = std.OrderBy(x => x.Name)
    });
    //Using Query Syntax
    var GroupByQS = (from std in Student.GetStudents()
    select std).ToLookup(x => x.Gender)
    //Then Sorting the data based on key in Descending
    .OrderByDescending(c => c.Key)
    .Select(std => new
    {
        Key = std.Key,
        //Sorting the Students in Each Group based on
Ascending order
        Students = std.OrderBy(x => x.Name)
    });

    //It will iterate through each groups
    foreach (var group in GroupByQS)
    {
        Console.WriteLine(group.Key + " : " + group.Students.Count());
        //Iterate through each student of a group
        foreach (var student in group.Students)
        {
            Console.WriteLine("  Name :" + student.Name + ", Age: " +
+ ", Branch :" + student.Branch);
        }
    }
    Console.Read();
}
}
}

```

Output:

```

Male : 5
  Name :Anurag, Age: 20, Branch :CSE
  Name :Pranaya, Age: 21, Branch :CSE
  Name :Sambit, Age: 21, Branch :ETC
  Name :santosh, Age: 22, Branch :CSE
  Name :Snurag, Age: 21, Branch :ETC
Female : 5
  Name :Celina, Age: 22, Branch :ETC
  Name :Hina, Age: 20, Branch :ETC
  Name :Preety, Age: 20, Branch :CSE
  Name :Priyanka, Age: 21, Branch :CSE
  Name :Tina, Age: 20, Branch :CSE

```

If you observe the above example, we have projected the result to an anonymous type. You can also create a new type with the required properties instead of an anonymous type and project the result to that new type. Let us understand this with an example. First, create a class file named **StudentGroup.cs** and copy and paste the following code.

```

using System.Collections.Generic;
namespace ToLookupDemo
{
    public class StudentGroup
    {
        public string Key { get; set; }
        public List<Student> Students { get; set; }
    }
}

```

With the above **StudentGroup** class in place, modify the Main method of the Program class as follows: Here, you can see we are projecting the result into the newly created StudentGroup type instead of the anonymous type.

```

using System;
using System.Linq;
namespace ToLookupDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            //First Group the Data by Gender using ToLookup Method
            var GroupByMS = Student.GetStudents().ToLookup(s => s.Gender)
                //Then Sorting the data based on key in Descending
                .OrderByDescending(c => c.Key)
                .Select(std => new StudentGroup
                {

```

```

        Key = std.Key,
        //Sorting the Students in Each Group based on
Ascending order
        Students = std.OrderBy(x => x.Name).ToList()
    });
//Using Query Syntax
var GroupByQS = (from std in Student.GetStudents()
    select std).ToLookup(x => x.Gender)
//Then Sorting the data based on key in Descending
.OrderByDescending(c => c.Key)
.Select(std => new StudentGroup
{
    Key = std.Key,
    //Sorting the Students in Each Group based on
Ascending order
    Students = std.OrderBy(x => x.Name).ToList()
});

//It will iterate through each groups
foreach (var group in GroupByQS)
{
    Console.WriteLine(group.Key + " : " + group.Students.Count());
    //Iterate through each student of a group
    foreach (var student in group.Students)
    {
        Console.WriteLine("    Name :" + student.Name + ", Age: " +
+ ", Branch :" + student.Branch);
    }
    Console.Read();
}
}
}
}

```

Run the application, and you will get the same output shown in the image below.

```

Male : 5
  Name :Anurag, Age: 20, Branch :CSE
  Name :Pranaya, Age: 21, Branch :CSE
  Name :Sambit, Age: 21, Branch :ETC
  Name :santosh, Age: 22, Branch :CSE
  Name :Snurag, Age: 21, Branch :ETC
Female : 5
  Name :Celina, Age: 22, Branch :ETC
  Name :Hina, Age: 20, Branch :ETC
  Name :Preety, Age: 20, Branch :CSE
  Name :Priyanka, Age: 21, Branch :CSE
  Name :Tina, Age: 20, Branch :CSE

```

Using LINQ ToLookup Method with Multiple Keys in C#:

Let us understand how to use the LINQ ToLookup Method in C# with Multiple Keys. We want to group the students first by Branch and then by Gender. For a better understanding, please look at the following example, which does exactly the same thing.

```

using System;
using System.Linq;
namespace ToLookupDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Grouping Students by Branch and Gender using ToLookup Method
            var GroupByMultipleKeysMS = Student.GetStudents()
                .ToLookup(x => new { x.Branch, x.Gender })
                .Select(g => new
                {
                    Branch = g.Key.Branch,
                    Gender = g.Key.Gender,
                    Students = g.OrderBy(x => x.Name)
                }); ;

            //It will iterate through each group
            foreach (var group in GroupByMultipleKeysMS)
            {
                Console.WriteLine($"Branch : {group.Branch} Gender: {group.Ger
Students = {group.Students.Count()}");
                //It will iterate through each item of a group
                foreach (var student in group.Students)
                {

```

```

        Console.WriteLine($" ID: {student.ID}, Name: {student.Nar
{student.Age} ");
    }
    Console.WriteLine();
}
Console.Read();
}
}
}
}

```

Output:

```

Barnch : CSE Gender: Female No of Students = 3
ID: 1001, Name: Preety, Age: 20
ID: 1006, Name: Priyanka, Age: 21
ID: 1008, Name: Tina, Age: 20

Barnch : ETC Gender: Male No of Students = 2
ID: 1010, Name: Sambit, Age: 21
ID: 1002, Name: Snurag, Age: 21

Barnch : CSE Gender: Male No of Students = 3
ID: 1004, Name: Anurag, Age: 20
ID: 1003, Name: Pranaya, Age: 21
ID: 1007, Name: santosh, Age: 22

Barnch : ETC Gender: Female No of Students = 2
ID: 1009, Name: Celina, Age: 22
ID: 1005, Name: Hina, Age: 20

```

Grouping Students Based on the Branch and Gender along with OrderBy Method

Now, we want to group the students first by Branch and then by Gender. First, we need to group the students by Branch in Descending Order, and then we need to group the students by Gender in Ascending order. Finally, the students in each group need to be sorted by their names in Ascending Order. For a better understanding, please look at the following example, which does exactly the same thing. The following example code is self-explained, so please go through the comment lines.

```

using System;
using System.Linq;
namespace ToLookupDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var GroupByMultipleKeysMS = Student.GetStudents()

```

```

Gender using ToLookup
Branch
Based on Gender
Ascending Order

//Group the Students first by Branch :
.ToLookup(x => new { x.Branch, x.Gender })
//Sort Each Group in Descending Order

.OrderByDescending(g => g.Key.Branch)
//Then Sort Each Branch Group in Ascending Order

.ThenBy(g => g.Key.Gender)
//Project the Result to an Anonymous Type
.Select(g => new
{
    Branch = g.Key.Branch,
    Gender = g.Key.Gender,
    //Sort the Students of Each group
    Students = g.OrderBy(x => x.Name)
});
//It will iterate through each group
foreach (var group in GroupByMultipleKeysMS)
{
    Console.WriteLine($"Branch : {group.Branch} Gender: {group.Gender}");
    Students = {group.Students.Count()});
    //It will iterate through each item of a group
    foreach (var student in group.Students)
    {
        Console.WriteLine($" ID: {student.ID}, Name: {student.Name}, Age: {student.Age} ");
    }
    Console.WriteLine();
}
Console.Read();
}
}
}

```

Output:

```
Barnch : ETC Gender: Female No of Students = 2
ID: 1009, Name: Celina, Age: 22
ID: 1005, Name: Hina, Age: 20

Barnch : ETC Gender: Male No of Students = 2
ID: 1010, Name: Sambit, Age: 21
ID: 1002, Name: Snurag, Age: 21

Barnch : CSE Gender: Female No of Students = 3
ID: 1001, Name: Preety, Age: 20
ID: 1006, Name: Priyanka, Age: 21
ID: 1008, Name: Tina, Age: 20

Barnch : CSE Gender: Male No of Students = 3
ID: 1004, Name: Anurag, Age: 20
ID: 1003, Name: Pranaya, Age: 21
ID: 1007, Name: santosh, Age: 22
```

What are the differences between ToLookup and GroupBy Methods in LINQ?

In LINQ (Language Integrated Query), both ToLookup and GroupBy methods are used to group elements from a collection based on a specified key selector function. However, they differ in their behavior and the type of results they produce:

Return Type:

- **GroupBy:** Returns an `IEnumerable<IGrouping<TKey, TElement>>`. Each `IGrouping<TKey, TElement>` object contains a collection of objects and a key. The groups are lazily evaluated, meaning the grouping isn't performed until you iterate over each group in the result.
- **ToLookup:** Returns an `ILookup<TKey, TElement>`. This is similar to a dictionary of keys, each mapped to one or more values. `ILookup` is eagerly evaluated, which means the grouping happens immediately when the method is called.

Execution:

- **GroupBy:** Uses deferred execution. The grouping is performed as you iterate over the result. This can be more efficient if you don't need to access all the elements of the group immediately.
- **ToLookup:** Uses immediate execution. The result is computed and stored in memory as soon as the method is called, making it faster for subsequent lookups.

Lookup Performance:

- **GroupBy:** Because the result of `GroupBy` is not immediately executed and stored in memory, accessing a group by its key is not as direct as with `ToLookup`. If you need to find a specific group by key, you would typically iterate over the result until you find the matching group.
- **ToLookup:** Since `ToLookup` results in a structure similar to a dictionary, it is more efficient for lookups by key. You can directly access a group using its key without having to iterate over the collection.

Use Cases:

- **GroupBy** is typically used when you need to group elements for immediate iteration, and you're concerned about memory usage, or when you're working with a large collection that might not fit into memory if evaluated eagerly.
- **ToLookup** is better suited when you need to perform multiple lookups by key on the grouped items or when you need all the groups to be immediately available.

When Should We Use the LINQ ToLookup Method in C#?

Here are scenarios when using ToLookup might be particularly useful:

- **Grouping Data:** When you have a collection of items you want to group by a specific key. ToLookup is handy because it groups items into a collection for each key immediately, allowing for immediate and multiple enumerations without re-querying the source.
- **Data Lookup Operations:** If your use case involves frequently looking up values by a key, ToLookup can be more efficient than using GroupBy, followed by a conversion to a dictionary or another lookup structure. With ToLookup, the grouped enumerable is immediately available for lookup by key.
- **Handling Duplicate Keys:** Unlike dictionaries, which cannot handle duplicate keys, a Lookup can store multiple values for a single key, making it ideal for datasets where keys are not unique.
- **Preventing Deferred Execution Issues:** Because ToLookup immediately executes the query and materializes the results into a Lookup object, it can help avoid common pitfalls related to deferred execution in LINQ, especially when the source data might change between different enumerations of the results.

Data Lookup Operations Example using LINQ ToLookup Method

A Lookup can store multiple values for a single key. It is useful when you want to group elements from a source sequence by a certain key while possibly having multiple values under each key. Here's an example to demonstrate how to use the ToLookup method. Imagine we have a list of books, with each book having a title and a genre. We want to group these books by their genre.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace GroupByDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            List<Book> books = new List<Book>
            {
                new Book("The Fellowship of the Ring", "Fantasy"),
                new Book("The Two Towers", "Fantasy"),
                new Book("The Return of the King", "Fantasy"),
                new Book("The Hobbit", "Fantasy"),
                new Book("Foundation", "Science Fiction"),
                new Book("Dune", "Science Fiction")
            };
        }
    }
}
```

```

var booksByGenre = books.ToLookup(book => book.Genre);
foreach (var genreGroup in booksByGenre)
{
    Console.WriteLine($"Genre: {genreGroup.Key}");
    foreach (Book book in genreGroup)
    {
        Console.WriteLine($"  \t{book.Title}");
    }
    Console.WriteLine();
}
Console.Read();
}
}
public class Book
{
    public string Title { get; set; }
    public string Genre { get; set; }
    public Book(string title, string genre)
    {
        Title = title;
        Genre = genre;
    }
}
}

```

In this example:

- We define a Book class with properties for Title and Genre.
- We create a list of Book objects.
- We use the ToLookup method to group the books by their Genre.
- Finally, we iterate over the groups and print out the genres and the titles of the books in each genre.

This example demonstrates how ToLookup can effectively group items by a key and handle multiple values for each key. This is especially useful in scenarios where an item belongs to a category with multiple items. When you run the above code, you will get the following output:

```

Genre: Fantasy
    The Fellowship of the Ring
    The Two Towers
    The Return of the King
    The Hobbit

Genre: Science Fiction
    Foundation
    Dune

```

Preventing Deferred Execution Issues Example using LINQ ToLookup Method

Deferred execution in LINQ is a concept where the query execution is delayed until the query result is actually iterated over. This feature is powerful as it allows for query composition and efficient execution.

However, it can sometimes lead to unexpected results, especially when the data source changes between the query definition and its execution. One way to avoid such issues is by using immediate execution methods like `.ToList()`, `.ToArray()`, or `.ToLookup()` at the point of the query where you want to ensure the data is fetched and stored, irrespective of any future changes to the underlying data.

The `.ToLookup()` method is particularly interesting because it allows you to immediately execute a query and organize the results into a `ILookup<TKey, TValue>`, which is a special kind of collection that allows for fast data retrieval by key. Here's a simple example that demonstrates how you can use `.ToLookup()` to prevent deferred execution issues:

Imagine you have a list of products, each with a category, and you want to group these products by category. You might initially consider using `.GroupBy()`, which uses deferred execution. However, if the list of products changes after you define your query but before you iterate over the results, you might have unexpected results. Using `.ToLookup()` instead can avoid this issue.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace GroupByDemo
{
    public class Program
    {
        public static void Main()
        {
            List<Product> products = new List<Product>
            {
                new Product { Name = "Apple", Category = "Fruit" },
                new Product { Name = "Banana", Category = "Fruit" },
                new Product { Name = "Cucumber", Category = "Vegetable" }
            };
            // Using ToLookup to immediately execute the query and group products by
            category
            var lookup = products.ToLookup(p => p.Category);
            // Imagine the products list changes here
            products.Add(new Product { Name = "Orange", Category = "Fruit" });
            // Iterating over the lookup
            foreach (var category in lookup)
            {
                Console.WriteLine($"Category: {category.Key}");
                foreach (var product in category)
                {
                    Console.WriteLine($"    Product: {product.Name}");
                }
            }
        }
    }
}
```

```

        Console.ReadKey();
    }
}
public class Product
{
    public string Name { get; set; }
    public string Category { get; set; }
}
}

```

In this example, even though we added “Orange” to the products list after creating the lookup, the output of the program will not include “Orange” because `.ToLookup()` executed immediately and created a snapshot of the data at that point in time. This ensures that the grouped results are consistent and unaffected by any changes to the underlying data source after the lookup is created. When you run the above code, you will get the following output:

```

Category: Fruit
Product: Apple
Product: Banana
Category: Vegetable
Product: Cucumber

```

Handling Duplicate Keys Example using Linq ToLookup Method

When working with collections in C#, the `ToLookup` method from LINQ (Language Integrated Query) is useful for handling duplicate keys efficiently. It groups elements of a collection by a specified key into an `ILookup<TKey, TElement>` structure, which is similar to a `Dictionary<TKey, TValue>`, but can store multiple values for a given key. Here’s a simple example to demonstrate how to use the `ToLookup` method to handle collections with duplicate keys:

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace GroupByDemo
{
    public class Program
    {
        public static void Main()
        {
            // Example list of employees
            List<Employee> employees = new List<Employee>
            {
                new Employee { Name = "John Doe", Department = "IT" },
                new Employee { Name = "Jane Smith", Department = "HR" },
                new Employee { Name = "Jack White", Department = "IT" },
                new Employee { Name = "Sara Parker", Department = "Finance" },
                new Employee { Name = "Tom Brown", Department = "IT" }
            };
            // Creating a lookup to group employees by department

```

```

var lookup = employees.ToLookup(emp => emp.Department);
// Displaying the grouped employees
foreach (var group in lookup)
{
    Console.WriteLine($"Department: {group.Key}");
    foreach (Employee emp in group)
    {
        Console.WriteLine($"  \t{emp.Name}");
    }
    Console.ReadKey();
}
}
public class Employee
{
    public string Name { get; set; }
    public string Department { get; set; }
}
}

```

In this example, a list of Employee objects is created, where each Employee has a Name and belongs to a Department. The ToLookup method is then used to group these employees by their department. This results in an `ILookup<string, Employee>` where the key is the department name, and the value is a collection of employees in that department.

The lookup object allows us to iterate over each group and its associated employees easily, even if there are multiple employees in the same department (demonstrating how it handles duplicate keys). This approach is particularly useful for grouping and querying collections without running into issues with duplicate keys. When you run the above code, you will get the following output:

```

Department: IT
    John Doe
    Jack White
    Tom Brown
Department: HR
    Jane Smith
Department: Finance
    Sara Parker

```

In the next article, I will discuss the [LINQ Joins in C#](#) with Examples. In this article, I explain the **LINQ ToLookup Method in C#** with Examples. I hope you understand the need for and use of the Linq ToLookup Method in C# with examples.